

AI Resume Generator Using FLAN-T5

JONNAGADLA DWARAKA NANDAN

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0138@dsu.edu.in

ENUGURU NIKESH

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0235@dsu.edu.in

KOLISETTY SUNDAR PAVAN

Department of CSE (AI-ML)
Dayananda Sagar University
Bangalore, India
eng23am0142@dsu.edu.in

Abstract—Resume writing demands language precision, domain-specific vocabulary, and the ability to articulate professional experiences concisely. Many candidates struggle to produce polished, well-structured resumes without external assistance. This paper presents an intelligent resume generation system that integrates four cooperative modules: a context-aware spell correction engine, a domain knowledge graph for skill recommendation, a FLAN-T5-based professional summary generator, and an AI-driven job description bullet point enhancer. The system operates on a structured dataset of 100 records spanning 50 feature columns, encompassing complete candidate profiles, spell-correction pairs, and skill-suggestion entries. FLAN-T5 (248 million parameters) is employed in a zero-shot prompting paradigm for natural language generation tasks, avoiding the computational overhead of fine-tuning. Evaluation across the dataset yields a spell correction accuracy of approximately 88%, skill suggestion coverage of approximately 85%, and human-evaluated quality scores of 88% for generated summaries and 85% for bullet point enhancement. An end-to-end pipeline connects spell correction, skill suggestions, AI text generation, and PDF rendering through a Flask web interface, producing downloadable resumes in under 20 seconds on CPU hardware. The proposed architecture demonstrates that lightweight, prompt-engineered transformer models can deliver practical, domain-specific natural language generation without task-specific training data.

Keywords — resume generation, FLAN-T5, prompt engineering, spell correction, skill knowledge graph, NLP, transformer, generative AI, professional summary, bullet point enhancement.

I. INTRODUCTION

The preparation of a professional resume represents one of the most consequential writing tasks a job seeker undertakes, yet it remains one of the least supported by intelligent tooling. Existing word processors provide grammar and spell-checking but offer no understanding of professional context, industry conventions, or the structural expectations of applicant tracking systems. Candidates who lack strong writing backgrounds or domain-specific vocabulary are frequently at a disadvantage, not because of deficient professional experience but because of limited ability to present that experience persuasively on paper. Generative AI, and large language models in particular, have demonstrated remarkable capability in instruction-following tasks that parallel the demands of resume writing: converting informal experience descriptions into polished prose, maintaining a consistent professional register, and generating text that is appropriately concise. However, deploying these capabilities in a practical resume tool raises several engineering challenges. Raw user input frequently contains typographical

errors, especially for technical terminology. The model must understand which skills are relevant to a given domain and be able to suggest complementary competencies. Generated text must be structured predictably enough to populate a formatted PDF template. This work addresses all three of these challenges within a unified pipeline. The system accepts user input through a web interface, applies spell correction to normalize technical vocabulary, queries a domain knowledge graph to surface relevant skill suggestions, invokes FLAN-T5 through structured prompts to generate professional summaries and bullet points, and renders the assembled content into one of five PDF templates via ReportLab. The contribution is not a single novel algorithm but rather a cohesive architecture that demonstrates how these components interact to produce a usable, end-to-end intelligent resume generation system. The following sections describe the dataset, the design of each module, the evaluation methodology, experimental results, and the overall system architecture.

II. LITERATURE REVIEW

Automated text generation for professional documents has attracted research attention from two converging directions: natural language generation for structured domains, and AI-assisted career support tools. Early work on controlled text generation relied on template-filling approaches in which pre-authored sentence skeletons were populated with domain-specific values [1]. While reliable and predictable, these systems lacked the linguistic flexibility needed to handle varied professional backgrounds and were brittle when user input deviated from expected formats. Neural sequence-to-sequence models introduced end-to-end generation from structured input representations. Encoder-decoder architectures trained on paired input-output data demonstrated the ability to produce fluent, contextually appropriate text across domains including abstractive summarization and dialogue generation [2]. These methods, however, depend on large paired training corpora that are not readily available for the resume domain, where privacy concerns limit data sharing. The advent of large-scale pretrained language models substantially reduced data requirements for downstream tasks. GPT-series models demonstrated that in-context examples could guide generation toward specific formats without gradient updates [3]. FLAN-T5, introduced by Google Research, extended this idea by fine-tuning the T5 encoder-decoder architecture on a diverse

collection of instruction-following tasks, producing a model that generalizes effectively to unseen task formats through natural language prompts alone [4]. This instruction-following capability makes FLAN-T5 particularly suitable for the resume generation setting, where the desired output format can be described precisely in the prompt. Spell correction for technical text presents challenges not captured by standard dictionary-based approaches. Technical terms — programming languages, framework names, and certification titles — are frequently absent from general-purpose word lists and may be incorrectly flagged or replaced. Hybrid approaches that combine statistical language models with domain-specific allowlists have demonstrated stronger performance on technical corpora [5]. Knowledge graph representations of skill relationships have been explored in labor market analytics for tasks including skill taxonomy construction and job-candidate matching [6]. Representing skills as graph nodes with edges to related competencies supports real-time recommendation at query time without requiring a retrieval model, making the approach practical for interactive web applications. PDF generation from structured data using Python libraries such as ReportLab has been established as an effective approach for template-driven document rendering in automated document processing systems [7]. The combination of programmatic layout control and platform-independent output makes this approach well-suited to multi-template resume generation.

III. DATASET AND FEATURES

A.. Structure and Composition The system is trained, tested, and evaluated against a purpose-built dataset of 100 records, each carrying 50 feature columns. Three record types are represented. Full profile records (30 rows) contain complete candidate information across personal details, education history, work experience, projects, certifications, and skill fields. Spell correction records (40 rows) pair raw text strings containing realistic typographical errors with their corrected counterparts, representing the kinds of mistakes users make when entering professional descriptions quickly. Skill suggestion records (30 rows) associate a seed skill with a set of related skills drawn from the domain knowledge graph, providing ground truth for evaluating suggestion coverage. **B.. Full Profile Features** Each full profile record contains 50 fields including name, role, years of experience, contact information, social profile URLs, a pipe-delimited skill string, a nested skill suggestion dictionary, educational qualifications at three levels (10th grade, intermediate, and degree), two certifications, two work experiences with raw descriptions and three pre-authored bullet points, one project entry with technology stack, a professional summary, a template preference, and a domain label. This rich feature structure enables end-to-end evaluation of the full pipeline. **C.. Spell Correction Pairs** Spell correction records contain a raw description field with intentional typographical errors — transposed characters, omitted vowels, and phonetic substitutions common in fast typing — alongside a corrected reference string. Forty such pairs span a range of professional domains including machine learning, DevOps, mobile devel-

opment, and data engineering. These pairs serve as the primary benchmark for spell correction accuracy measurement. **D.. Skill Suggestion Records** Skill suggestion records each contain a single seed skill in the skills field and a corresponding list of expected related skills drawn from the domain knowledge graph. Thirty records cover major technical domains. The expected related skills serve as ground truth against which the suggestion module’s output is compared.

METHODOLOGY

A. Overview

The system follows a sequential pipeline architecture with six functional stages. User input from a web form passes through spell correction, then skill suggestion, then AI text generation for the summary and bullet points, then education and project structuring, and finally PDF rendering and delivery. Each stage operates on the output of the previous one, and the entire pipeline executes within a single `build_resume()` function that orchestrates the flow.

B. Spell Correction Module

The spell correction module uses the `pyspellchecker` library as its base engine, augmented with a custom allowlist of 60 technical terms covering programming languages, frameworks, cloud platforms, and certification names. Before any correction is attempted, the module checks whether a token satisfies any of three preservation rules: the token is entirely uppercase (such as SQL or AWS), the token is CamelCase (such as TensorFlow or PyTorch), or the token appears in the custom allowlist. Tokens that meet any of these criteria are passed through unmodified. Remaining tokens are submitted to the `pyspellchecker` distance-one correction algorithm, which selects the most probable correction based on unigram frequency in the underlying word list.

This hybrid design addresses a fundamental limitation of general-purpose spell checkers when applied to technical text: standard dictionaries do not contain most technology names, and naive correction would replace established framework names with unrelated common words. The custom allowlist ensures that domain vocabulary is preserved exactly as intended.

C. Skill Knowledge Graph

The skill recommendation module encodes domain expertise as a static directed knowledge graph implemented as a Python dictionary. Each key represents a skill or technology, and its associated value is an ordered list of related skills ranked by relevance. The graph covers 30 technical domains including artificial intelligence, web development, DevOps, mobile development, cybersecurity, data engineering, and database management, with several hundred skill-to-skill relationships in total.

Two public interfaces expose the graph. The `suggest_typing()` function accepts a partial query string and returns up to `n` matching skills, enabling dropdown autocomplete as the user types. The `suggest_added()` function accepts the list

of skills already entered and returns ranked related skills not yet in the current skill set, supporting chip-style recommendation. Both functions operate at dictionary lookup speed, adding negligible latency to the user interaction.

D. FLAN-T5 Text Generation

The generation backbone is google/flan-t5-base, loaded from Hugging Face with 248 million parameters and used exclusively for inference with no gradient computation. A single generate() wrapper function handles tokenization, beam search decoding with beam width 4, and detokenization. Maximum output token count is configurable per call.

Professional Summary Generation: The generate_summary() function constructs a structured prompt specifying the candidate’s name, role, years of experience, and skill list, instructing the model to produce exactly two professional sentences in third-person present tense. Post-processing normalizes punctuation and whitespace.

Bullet Point Enhancement: The enhance_bullets() function applies spell correction to the raw job description, then constructs a prompt instructing the model to convert the experience into exactly three resume bullet points, each beginning with a strong past-tense action verb and including quantified results where possible. Output lines are parsed by splitting on newlines and retaining lines longer than 15 characters. Up to three valid bullets are returned.

E. PDF Generation and Web Interface

The PDF generation module uses ReportLab to render resume content into one of five templates: modern, classic, executive, creative, and minimal. Each template defines its own color scheme, font choices, section ordering, and layout geometry. The web interface is implemented with Flask and provides a single-page form collecting all resume fields. On submission, the form data is assembled and processed by build_resume(), and the rendered PDF is returned as a download response.

F. Complete Pipeline Flow

The build_resume() function executes five sequential steps: (1) spell correction of name, role, location, and all skill tokens; (2) AI professional summary generation via FLAN-T5; (3) AI bullet point enhancement for each work experience entry; (4) structuring of education entries; and (5) assembly of the final resume dictionary with project, certification, template, and color fields, which is passed directly to the ReportLab renderer.

IV. EVALUATION

A. Spell Correction Accuracy

Spell correction accuracy is measured against the 40 spell correction records in the dataset. For each record, the raw description field is passed through the correct_spelling() function and the output is compared case-insensitively against the reference corrected string. The resulting accuracy is approximately 88%, reflecting strong performance on common typographical

patterns while acknowledging that highly unusual or novel errors may fall outside the correction model’s vocabulary.

B. Skill Suggestion Coverage

Skill suggestion coverage is measured across the 30 skill suggestion records. For each record, the seed skill is passed to both suggest_typing() and suggest_added() with n set to 12, and the union of both result sets is compared against the expected related skills. Coverage per record is the fraction of expected skills appearing in the combined output, and the mean across all records is reported. The system achieves approximately 85% coverage, with the primary gap attributable to highly specialized sub-domain skills not yet represented in the current graph.

C. Summary and Bullet Quality

AI-generated summaries and bullet points are evaluated through human assessment rather than automated metrics, since standard measures such as BLEU are poorly suited to open-ended professional text generation. Reviewers assessed generated outputs against three criteria: grammatical correctness, professional register, and content relevance to the input. Summaries achieved approximately 88% quality and bullet points approximately 85%. Common failure modes included occasional repetition of input phrases and generation of generic rather than specific bullet content when the raw description lacked quantifiable detail.

D. Latency

On CPU hardware, the complete build_resume() pipeline executes in approximately 15 to 20 seconds per resume, with the majority of time consumed by the two FLAN-T5 generation calls. GPU execution reduces this to under 5 seconds. Both figures are within acceptable bounds for a web application where users are informed that generation is in progress.

V. RESULTS AND DISCUSSION

TABLE I
SYSTEM PERFORMANCE EVALUATION

Module	Metric	Score
Spell Correction	Accuracy vs. reference strings	~ 88%
Skill Suggestion	Coverage of expected related skills	~ 85%
Summary Generation	Human evaluation quality	~ 88%
Bullet Enhancement	Human evaluation quality	~ 85%
Full Pipeline (CPU)	End-to-end generation time	15–20 sec

TABLE II
DATASET DISTRIBUTION

Record Type	Row Count	Primary Use
full_profile	30	End-to-end pipeline testing
spell_correction	40	Spell correction accuracy evaluation
skill_suggestion	30	Skill suggestion coverage evaluation
Total	100	All modules

The experimental results confirm that a zero-shot prompting strategy with FLAN-T5 is sufficient to produce professionally

TABLE III
MODULE TECHNOLOGIES AND FUNCTIONS

Module	Tech	Function
Spell Correction	pyspellchecker + allowlist	Normalizes input
Skill Graph	Static dictionary	Typing suggestions
Summary Gen	FLAN-T5 (zero-shot)	2-line summary
Bullet Enhancer	FLAN-T5	3 bullet points
PDF Renderer	ReportLab	PDF output
Web Interface	Flask + HTML/CSS	User form

appropriate resume content for the majority of input profiles, without any task-specific fine-tuning. This has significant practical implications: the system can be deployed without labelled training data for the generation task, and the prompts that drive generation can be revised iteratively as requirements evolve without any retraining overhead.

The spell correction module’s 88% accuracy reflects both the strength of the hybrid allowlist approach and its principal limitation. The allowlist is manually curated and has a fixed vocabulary. Technical terms introduced after the allowlist was assembled — such as newly released frameworks or emerging cloud services — will not be recognized and may be incorrectly altered. A dynamic allowlist that ingests new technical vocabulary from a curated external source would address this gap.

The skill knowledge graph’s 85% suggestion coverage demonstrates that a static graph with 30 domain nodes can cover a substantial fraction of realistic skill relationships. The graph’s primary weakness is its uniform treatment of relationship strength: all related skills for a given node are returned in list order rather than weighted by contextual relevance. Incorporating edge weights derived from co-occurrence statistics in job posting datasets would likely improve both precision and coverage.

The gap between human-evaluated quality scores for summaries (88%) and bullet points (85%) reflects the greater structural constraint placed on the bullet generation task. The model must produce exactly three distinct bullets, each beginning with a past-tense action verb, within a token budget that may not accommodate all the information in a verbose raw description. The summary task has more degrees of freedom and benefits from the model’s stronger general capability for fluent prose.

One architectural consideration that warrants future investigation is the handling of very long work descriptions. The current implementation applies a fixed max_tokens limit, which may truncate output for candidates with extensive experience records. An adaptive token budget based on input length would produce more complete output in these cases.

VI. CONCLUSION

This paper has described an end-to-end AI resume generation system built around four cooperating modules: context-aware spell correction, domain knowledge graph-driven skill suggestion, and FLAN-T5-based zero-shot generation of professional summaries and bullet points, all integrated into a

Flask web application with ReportLab PDF rendering. Evaluation against a structured 100-record dataset demonstrates spell correction accuracy of approximately 88%, skill suggestion coverage of approximately 85%, and human-evaluated generation quality of approximately 88% for summaries and 85% for bullet points, with complete pipeline execution under 20 seconds on CPU hardware.

The system demonstrates that instruction-tuned encoder-decoder transformers are well-suited to professional document generation tasks when paired with targeted pre-processing and domain knowledge representations. The zero-shot approach removes the dependency on task-specific labelled training data, making the system straightforward to adapt to new professional domains by modifying prompts and extending the knowledge graph rather than retraining the model.

Future work will explore fine-tuning FLAN-T5 on curated resume datasets to improve output specificity, incorporating dynamically updated skill graphs derived from live job market data, and extending the template library to support industry-specific resume formats in domains such as academia and creative industries.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Dayananda Sagar University for providing the necessary infrastructure, computing resources, and a supportive learning environment that enabled us to successfully carry out this research work.

We also extend our appreciation to the developers and contributors of open-source technologies that played a crucial role in this project. In particular, we acknowledge the contributions of the Hugging Face Transformers library for enabling the use of the FLAN-T5 model, and the developers of PySpellChecker for efficient text correction.

These tools and frameworks were instrumental in the development and implementation of our AI-based Resume Generator system.

REFERENCES

- [1] E. Reiter and R. Dale, *Building Natural Language Generation Systems*. Cambridge, UK: Cambridge University Press, 2000.
- [2] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 3104–3112.
- [3] T. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [4] H. W. Chung *et al.*, "Scaling Instruction-Finetuned Language Models," arXiv preprint arXiv:2210.11416, 2022. [Online]. Available: <https://arxiv.org/abs/2210.11416>
- [5] M. D. Yannakoudakis and R. Fawthrop, "An Intelligent Spelling Error Corrector," *Information Processing and Management*, vol. 19, no. 2, pp. 101–108, 1983.
- [6] M. Khauja, I. Kassou, and M. Ghogho, "A Survey on Text-Based Automatic Personality Prediction," *IEEE Access*, vol. 9, pp. 156706–156727, 2021.
- [7] ReportLab Inc., "ReportLab PDF Library User Guide," ReportLab, London, UK, 2023. [Online]. Available: <https://www.reportlab.com/documentation/>